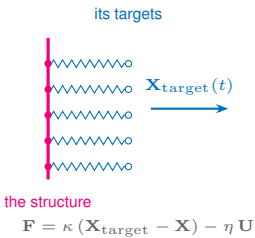
# TARGET POINTS, SPRINGS AND MULTIPLE BOUNDARIES

Prescribing motion by moving target points, changing springs in time, and running several structures at once

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 9a




Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

## LEARNING GOALS

By the end of this chapter you should be able to:

1. say what a target point and a spring do inside IBAMR (the force laws, where their parameters live at run time, which point "owns" a spring);
2. write a function that moves target points along a prescribed path, call it from `main.cpp` before each time step, and build the program with the extra source file;
3. write the same kind of function for springs: change rest lengths or stiffnesses in time, correctly (through a reference, not a copy);
4. run several structures in one simulation, find each one's range of point indices, and move them differently;
5. read the three examples that do this (IB/04 moving cylinders, IB/06 growing springs, IB/07 moving plates), check that a prescribed motion is being followed, and recognize the mistakes that make it silently fail.

> Chapter 9b continues with forces you write yourself (custom spring laws, applied external forces, active pumping). Chapter 10 is beams and curvature. Chapter 6 has the file formats; chapter 8a the stiffness estimates; chapter 11 the forces that come out.

# HOW THIS CHAPTER WAS CHECKED

- Every class and function named here was checked against the IBAMR 0.19.0 headers (`IBTargetPointForceSpec.h`, `IBSpringForceSpec.h`, `LDataManager.h`, `LMesh.h`, `LNode.h`); the 2012–2020 lectures' older spellings are listed on slide 16 so that old code can be read.
- The three examples (IB/04, IB/06, IB/07) build with 0 warnings on IBAMR 0.16.0 and 0.19.0 and ran to the end; the figures are from their shipped runs, analysed from the `hier_data` position files with `tools/ch9a_figures.py`.
- The one real bug in the course material (a rest length changed on a copy, so the springs never grew) is explained on slide 19 and fixed in IB/06; the READMEs record the measured time-step limits and the motions.
- The 2015–2020 statements about multiple boundaries ("I don't think you can connect springs between them") are checked against the 0.19.0 initializer on slide 28.

> Slide numbers refer to this deck; chapter numbers (3, 5, 6, 7, 8a, 8b, 9b, 10, 11, 13, 16) to the other chapters of this series.

# WHERE TO LOOK THINGS UP

- The header of `LDataManager` (`ibtk/include/ibtk/LDataManager.h`) carries, in its documentation comment, a complete worked example of moving target points and one of reading the points' positions: the modern reference for everything in this chapter (slides 11, 13).
- The force-spec headers: `include/ibamr/IBTargetPointForceSpec.h`, `IBSpringForceSpec.h`, `IBBeamForceSpec.h`: what each object stores and the accessors that return references (slide 5).
- The examples of this series: IB/04 (one structure, moving targets), IB/07 (two structures, moving targets), IB/06 (springs changed in time); each has its `update_*.cpp`, `.h`, the `main.cpp` hook and a README with the measured facts.
- The IBAMR FAQ on the wiki, https://github.com/IBAMR/IBAMR/wiki/FAQ: the 2012 answers quoted on slide 7 came from its predecessor.
- The 2013 slides pointed to the Google Code source browser and class pages, and to course servers (sakai, nemo); all gone. The class documentation is now generated from the headers (https://ibamr.github.io/docs).

# TARGET POINTS AND SPRINGS, AS IBAMR SEES THEM AT RUN TIME

The files of chapter 6 are read once, at start-up, by `IBStandardInitializer`. From then on each element lives in a small object attached to one Lagrangian point, its master point:

- `IBTargetPointForceSpec`: for a target point, its stiffness $\kappa$, damping $\eta$ and target position $\mathbf{X}_{\text{target}}$ (initially the point's own position). Force $\kappa\,(\mathbf{X}_{\text{target}} - \mathbf{X}) - \eta\,\mathbf{U}$: "a penalty force that approximately imposes a Dirichlet boundary condition at a single node" (the header).
- `IBSpringForceSpec`: for the lower-numbered end of each spring, the list of its springs: the other end's index, the force-function index, and a parameter vector per spring, by convention `[0]` the stiffness and `[1]` the rest length. "Springs now have an arbitrary number of parameters" (2020).
- `IBBeamForceSpec`: the neighbour pairs, bending rigidities and preferred curvatures (chapter 10).

```
// IBTargetPointForceSpec (0.19.0): the accessors return
  references
int&          getMasterNodeIndex();
double&       getStiffness();
double&       getDamping();
IBTK::Point& getTargetPointPosition();

// IBSpringForceSpec
unsigned int                        getNumberOfSprings() const
  ;
int&                                getMasterNodeIndex();
std::vector<int>&                   getSlaveNodeIndices();
std::vector<int>&                   getForceFunctionIndices();
std::vector<std::vector<double>>& getParameters();   // [k
  ][0] stiffness, [k][1] rest length
```

Because they return references, you can change a target position or a rest length in place and the force routine (`IBStandardForceGen`, chapter 8b) uses the new value at the next force evaluation. That is the whole mechanism of this chapter. "You may change the parameter values but must not resize the vectors" (the header of `getParameters`).

# THREE WAYS TO PRESCRIBE MOTION

| Way | How | Use it for |
|---|---|---|
| move the target points (this chapter) | before each step, shift every $\mathbf{X}_{\text{target}}$ along the path; the tethers drag the structure after them | a rigid or nearly rigid body on a known path: a translating plate, a rotating wing, a pumping wall, a flapping fin (with soft tethers it also bends) |
| change the springs (this chapter) | before each step, change rest lengths (growth, muscle shortening) or stiffnesses (activation) | structures that change shape from inside: growing fibres, contracting tubes, stiffening or softening tissue |
| apply a force (chapter 9b) | write a force law or an external body force and register it | muscles with a force–length law, pumping by travelling waves of force, anything that is not a spring |

All three happen *on the Lagrangian side*: the fluid still sees only a force density spread by the kernel (chapter 5). A moving target point does not set a velocity; it sets a force that pulls the point toward where it should be, so the structure lags its path by $F/\kappa$ and the tether force is the force the fluid exerts on the structure (slide 15).

The 2014–2020 “What can go wrong” advice applies to all three: ramp the motion from rest (chapter 7), and do not make the stiffness larger than the time step can take (slide 33).

# THE 2012 FAQ ANSWERS, KEPT WORD FOR WORD

*How can I have time-dependent target point locations?* "Take a look at the `moving_plates` example. This example is for the specific case of time-dependent target point locations. The linked code has a routine (`update_target_point_positions`) that loops over the local IB points (i.e., the IB points that are 'local to the processor'). The pseudo-code is roughly:"

```
for each local IB point {
    check to see if the point has a target point spec
    if yes: update the material properties w/ that target spec
}
```

"The properties for the target points are stored in `IBTargetPointForceSpec` objects. Functionality has been added to the most recent IBAMR versions to simplify the code required to access these objects. In particular, for each Lagrangian (IB) point, you can do `tbox::Pointer<T> spec = node_idx.getStashData<T>();` to get a pointer to any object of type T that is associated with the Lagrangian point."

Today's spelling is `node->getNodeDataItem<T>()`, returning a plain pointer (slide 16); everything else in this answer is still exactly right. The springs' answer is on the next slide.

# THE 2012 FAQ ANSWERS, CONTINUED: SPRINGS

*How can I have time-dependent spring constants or resting lengths?* "If you use class `IBStandardInitializer` to initialize the Lagrangian structure and provide a `.spring` file, then the first point in each spring will have associated with it an object of type `IBSpringForceSpec`. Each spring is associated with ONLY ONE of the two ends of the spring. By default, this is the 'master' point index, which is simply the lower of the two indices. This means that it is possible for some IB points not to have any force spec objects, even though those points might be connected to another point by a spring (or a beam). If there are not any objects of a specified type T associated with a particular node, then `getStashData<T>()` will return a null pointer. You should always check to see if the returned pointer is non-null before trying to do anything with it."

"To update the properties of the objects that describe the springs will require a function similar to the `update_target_point_positions` function. Refer to the documentation for class `IBSpringForceSpec` to see what properties you can access and modify."

Still exactly right, with `getNodeDataItem` for `getStashData` and `getParameters()` for the properties (slides 5, 18). The one thing the answer does not say, because it was not yet a problem, is that the property must be changed *in place* (slide 19).

# WHERE THE UPDATE IS CALLED: THE TIME LOOP IN `main.cpp`

```
// IB/04 main.cpp, inside the time loop (condensed)
while (!MathUtilities<double>::equalEps(loop_time, loop_time_end) &&
       time_integrator->stepsRemaining())
{
    iteration_num = time_integrator->getIntegratorStep();
    loop_time     = time_integrator->getIntegratorTime();
    dt = time_integrator->getMaximumTimeStepSize();

    // Move the cylinders' target points to where they should be at the
    // end of this time step (update_target_point_positions.cpp).
    LDataManager* l_data_manager = ib_method_ops->getLDataManager();
    update_target_point_positions(patch_hierarchy, l_data_manager,
  loop_time, dt);

    time_integrator->advanceHierarchy(dt);
    loop_time += dt;
    ...
}
```

```
# CMakeLists.txt: both source files, or the link step
    fails
add_executable(main2d main.cpp
  update_target_point_positions.cpp)
```

The program is IB/01's (chapter 3) plus one call per step, placed before `advanceHierarchy`: the targets are moved to where they should be at the end of the step, and the step then pulls the structure after them.

- `ib_method_ops->getLDataManager()` hands over the object that owns the Lagrangian data on every level.
- `loop_time` is the time at the start of the step and `dt` the step the integrator is about to take; the function receives both.
- "You also need to add these files to `main.c`" (2015–2020): today that is the `#include` of the header in `main.cpp` and the extra file in `add_executable`. Leave it out and the link fails with `undefined reference to` `` `update_target_point_positions'. ``

An alternative that needs no edit of the loop: register the function as a callback with `time_integrator->registerPreprocessIntegrateHierarchyCallback(...)` (`HierarchyIntegrator.h`); the examples keep the explicit call because it is easier to read.

# THE FUNCTION'S SIGNATURE AND WHAT YOU GET

```
// update_target_point_positions.h (IB/04; the whole header)
#ifndef included_update_target_point_positions
#define included_update_target_point_positions
// IBAMR 0.16 needs LData.h for LDataManager.h to compile; 0.19
  does not.
#include <ibtk/LData.h>
#include <ibtk/LDataManager.h>
#include <PatchHierarchy.h>

void update_target_point_positions(
    SAMRAI::tbox::Pointer<SAMRAI::hier::PatchHierarchy<NDIM> >
  hierarchy,
    IBTK::LDataManager* l_data_manager,
    double current_time,
    double dt);
#endif
```

"You have access to the `current_time` and time step (`dt`). You also have access to the patch hierarchy and can get the nodes the particular processor is working on." (2015–2020)

The four arguments:

- `hierarchy`: the grid; needed only for `getFinestLevelNumber()`, because the structure lives on the finest level (the `level_number` of chapter 6);
- `l_data_manager`: gives the Lagrangian mesh of a level (`getLMesh`), the index range of each structure (`getLagrangianStructureIndexRange`) and the position data (`getLData("X", ln)`);
- `current_time`, `dt`: the path is a function of time, and the increment to add this step is (speed at this time) $\times$ `dt`.

The course header included about twenty IBAMR, SAMRAI and PETSc headers, among them `IBAMR_config.h` and `IBTK_config.h`, which CMake installs do not have, and `app_namespaces.h`, which IBAMR says must never be included from a header. The three above are all it needs (chapter 3's porting checklist).

# THE LOOP OVER THE POINTS

```
const int finest_ln = hierarchy->getFinestLevelNumber();

// Structure 0's index range [first, second)
const std::pair<int, int> cyl_lag_idxs =
    l_data_manager->getLagrangianStructureIndexRange(0, finest_ln);

// The Lagrangian mesh of the finest level: local nodes AND ghost nodes
Pointer<LMesh> mesh = l_data_manager->getLMesh(finest_ln);
std::vector<LNode*> nodes = mesh->getLocalNodes();
nodes.insert(nodes.end(), mesh->getGhostNodes().begin(), mesh->
  getGhostNodes().end());

for (LNode* const node : nodes)
{
    IBTargetPointForceSpec* const force_spec = node->getNodeDataItem<
  IBTargetPointForceSpec>();
    if (force_spec == nullptr) continue;      // not a target point

    const int lag_idx = node->getLagrangianIndex();
    Point& X_target = force_spec->getTargetPointPosition();
    if (cyl_lag_idxs.first <= lag_idx && lag_idx < cyl_lag_idxs.second)
    {
        X_target[1] += (something) * dt;      // the motion goes here
    }
}
```

Line by line (the 2020 lecture's comments, updated):

- `getLagrangianStructureIndexRange(0, ln)`: "the first point on the structure; if you have more than one boundary, you can use this to determine which boundary you are on." Half-open: `second` is one past the last index.
- `getLMesh`, `getLocalNodes`, `getGhostNodes`: the points this process owns and the copies it holds of neighbours' points. "In parallel, we also need to update the target point locations of the ghost data" (the header's example).

The rest of the loop is on the next slide.

# THE LOOP OVER THE POINTS, CONTINUED

```
for (LNode* const node : nodes)
{
    IBTargetPointForceSpec* const force_spec = node->
  getNodeDataItem<IBTargetPointForceSpec>();
    if (force_spec == nullptr) continue;      // not a target
  point

    const int lag_idx = node->getLagrangianIndex();
    Point& X_target = force_spec->getTargetPointPosition();
    if (cyl_lag_idxs.first <= lag_idx && lag_idx < cyl_lag_idxs.
  second)
    {
        X_target[1] += (something) * dt;      // the motion goes
  here
    }
}
```

Why the ghosts matter: leave them out and a point near a patch boundary keeps its old target on one process while its copy on the neighbouring process moves. On one process nothing is wrong; in parallel the structure tears or lags at process boundaries, and the symptom appears only on the cluster.

- `getNodeDataItem<T>()`: “get data from the target point, such as its position”; `nullptr` if the point has no target spec, which is why the test comes first.
- `getLagrangianIndex()`: the point’s number in the files (chapter 6), fixed for the run; the PETSc index, which changes at every regrid, is a different thing (slide 13).
- `Point& X_target`: a reference into the spec; `X_target[0]`, `[1]`, `[2]` are its components, and adding to them changes the stored target.
- the range test lets one function serve several structures (slide 23); with one structure it is always true, and the 2020 lecture kept it “because we only have one wing in this case”.
- `(something) * dt`: a speed times the step, so that the targets move by the right distance whatever `DT` is.

# READING THE POINTS' CURRENT POSITIONS IN CODE

```
// from the LDataManager.h documentation (0.19.0), condensed
void print_eul_data(Pointer<PatchHierarchy<NDIM>> hierarchy,
  LDataManager* const l_data_manager)
{
    const int finest_ln = hierarchy->getFinestLevelNumber();
    Pointer<LData> X_data = l_data_manager->getLData("X", finest_ln);
    Vec X_vec = X_data->getVec();
    double* x_vals;
    int ierr = VecGetArray(X_vec, &x_vals);
    IBTK_CHKERRQ(ierr);

    Pointer<LMesh> mesh = l_data_manager->getLMesh(finest_ln);
    for (const auto& node : mesh->getLocalNodes())
    {
        const int lag_idx   = node->getLagrangianIndex();
        const int petsc_idx = node->getLocalPETScIndex();   // local,
  not global
        Eigen::Map<VectorNd> X(&x_vals[petsc_idx * NDIM]);
        pout << "Eulerian location of node " << lag_idx << ":\n" << X
  << "\n";
    }
    // (restore the array with VecRestoreArray when done)
}
```

A target point knows where it *should* be; the point's actual position lives in the `LData` vector `"X"`, a PETSc vector in PETSc ordering. From the header: "The Lagrangian ordering is fixed at the beginning of the simulation and is set by the reading of the vertices. The PETSc ordering, however, will change over the course of the simulation. Whenever a regridding operation is triggered, the PETSc indexing is changed to ensure efficient memory usage."

- so a position is reached through the node's `getLocalPETScIndex()`, never through its Lagrangian index;
- `"X"` is the position, `"U"` the velocity, `"F"` the force, on the same ordering;
- uses: a tether law that depends on where the point *is* (chapter 9b), a feedback controller, or a printed check of the lag (slide 34).

For analysis after the run, the `X.*` files of chapter 11 are simpler: they are already in Lagrangian order.

# THE MOVING CYLINDERS (IB/04): THE MOTION

```
static const double V = 0.2;            // speed after the ramp (m/s)
static const double time_to_acc = 0.1; // time to accelerate from 0
  to V (s)
...
    if (cylinder_lag_idxs.first <= lag_idx && lag_idx <
  cylinder_lag_idxs.second)
    {
        if (current_time <= time_to_acc)
        {
            X_target[1] += current_time / time_to_acc * V * dt; //
  speeding up
        }
        else
        {
            X_target[1] += V * dt;                              //
  constant speed
        }
    }
```

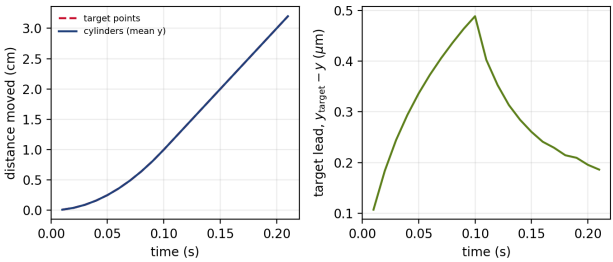
The 2015–2020 lecture's gloss: "check to see if we are in the acceleration phase; check to see if the node is on the first wing (it is, because we only have one wing in this case); determine the new position in the $y$-direction, we aren't moving in the $x$-direction here; here we move the wing at a constant speed."

What the code says:

- the targets move in $+y$ at a speed that rises linearly from $0$ to $V$ over `time_to_acc`, then stays at $V$: a ramp, for the reason of chapter 7 (no impulsive start);
- the increment is a displacement, speed $\times$ `dt`, added to the target each step; the targets are never *set* to a position, only shifted, so the function needs no memory of where they started;
- the course comments called the row "the wing" and said it moves "to the left or right" (it moves up); the series' comments say what the code does.

The structure: 9 cylinders of radius 2.5 mm (5 cells across), 33 points each, 0.05 m apart on the $x$ axis, every point a target with $\kappa = 4.1 \times 10^6$; $\mu = 100$, so $\mathrm{Re} = 0.01$ (chapters 8a, 8b). The README calls them bristles; the course called the row a holey plate.

# THE MOVING CYLINDERS: DO THEY FOLLOW?




From the shipped run (0.2 s, 4 cores): the mean $y$ of the 297 points against the target path. The two curves lie on top of each other; the right panel is their difference.

- the lead of the targets peaks at $0.48$ $\mu$m at the end of the ramp and settles to $0.2$ $\mu$m: a five-thousandth of a cell ($\Delta x = 1/1024$ m);
- the lead is the force the fluid exerts on the row divided by $\kappa$: with $\kappa = 4.1 \times 10^6$ per point, a drag of a fraction of a newton per metre per point is a fraction of a micrometre. `integral{F}` prints that force (chapter 11): $249.6$ N/m in $y$ at the end of the run;
- the peak at $t = 0.1$ s is the acceleration reaction: the fluid resists the change of speed, not the speed.

The price of following this closely is the time step: $\kappa = 4.1 \times 10^6$ needs `DT = 2e-5` (slide 33).

IB/04 with IBAMR 0.19.0, shipped input; `tools/ch9a_figures.py` reads `hier_data_IB2d/X.*` and the `.vertex` file (the positions at $t = 0$).

# ONE API ACROSS FOUR VERSIONS

The 2012–2020 slides show the same function in three dialects, and the comments say “depending on the version of IBAMR, you need to select one of the ways”. The spellings, oldest first:

| What | 2009–2013 (Griffith’s `moving_plates`, the 2012 deck) | 2015–2020 courses → 0.19.0 (this series) |
|---|---|---|
| iterate over the points | `LNodeIndexData` on each patch: `for (PatchLevel::Iterator p(level); ...) ... idx_data->lnode_index_begin(patch_box)` | `mesh->getLocalNodes()` plus `getGhostNodes()` from `l_data_manager->getLMesh(ln)` |
| the point | `const LNodeIndex& node_idx = *it` | `LNode* node` |
| the spec | `tbox::Pointer<T> spec = node_idx.getStashData<T>()`; test `!spec.isNull()` | `T* spec = node->getNodeDataItem<T>()`; test `spec != nullptr` |
| the target position | `std::vector<double>& X_target`, then `TinyVector<double,NDIM>&`, then `IBTK::Vector<double,NDIM>&` | `IBTK::Point& X_target = spec->getTargetPointPosition()` (an Eigen vector) |
| a spring’s rest length | `spec->getRestingLengths()[k]`, stiffness `getStiffnesses()[k]` | `spec->getParameters()[k][1]`, stiffness `[k][0]` |
| source file | `update_target_point_positions.C`, Makefile | `.cpp`, CMake (chapter 3) |

Old code found on a disk or in a thesis is worth reading for its *motion*; port the mechanics with this table and chapter 3’s checklist. The 2009 version (next slide, and in full in the archive, Part X) did two things the course versions later fixed: it did not ramp the speed (the plates started at $V = 0.1$ m/s instantly), and it did not visit ghost nodes (its patch iteration covers only the interior of each patch).

# THE OLDEST VERSION (2009), FOR READING OLD CODE

```
// Created on 06 Oct 2009 by Boyce Griffith; shown in the January 2012 tutorial
const int lag_node_index_idx = lag_manager->getLNodeIndexPatchDescriptorIndex();
tbox::Pointer<hier::PatchLevel<NDIM> > level = hierarchy->getPatchLevel(finest_ln);
for (SAMRAI::hier::PatchLevel<NDIM>::Iterator p(level); p; p++)
{
    SAMRAI::tbox::Pointer<SAMRAI::hier::Patch<NDIM> > patch = level->getPatch(p());
    const SAMRAI::hier::Box<NDIM>& patch_box = patch->getBox();
    const SAMRAI::tbox::Pointer<LNodeIndexData> idx_data = patch->getPatchData(lag_node_index_idx);
    for (LNodeIndexData::LNodeIndexIterator it = idx_data->lnode_index_begin(patch_box);
         it != idx_data->lnode_index_end(); ++it)
    {
        const LNodeIndex& node_idx = *it;
        const int lag_idx = node_idx.getLagrangianIndex();
        SAMRAI::tbox::Pointer<IBTargetPointForceSpec> force_spec = node_idx.getStashData<
  IBTargetPointForceSpec>();
        if (!force_spec.isNull())
        {
            std::vector<double>& X_target = force_spec->getTargetPointPosition();
            if (plate2d_left_lag_idxs.first <= lag_idx && lag_idx < plate2d_left_lag_idxs.second)
  X_target[0] -= dt*V;
            if (plate2d_rght_lag_idxs.first <= lag_idx && lag_idx < plate2d_rght_lag_idxs.second)
  X_target[0] += dt*V;
        }
    }
}
```

The same algorithm as slide 11: find the index ranges, visit every point, test for a target spec, shift the target. The differences are all in how the points are reached (patch by patch, through the `LNodeIndexData` patch data) and in the smart-pointer types.

The 2012 deck also quoted the file formats of chapter 6 as they then were: “the rest length and force function index are optional; by default the rest length is 0.0” was true then, and is not now (chapter 6: the rest length is required).

# CHANGING SPRINGS IN TIME: THE GROWING FIBRE (IB/06)

```
// update_springs.cpp (IB/06), the loop
static const double growth_rate = 0.01;   // how fast each rest
  length grows (m/s)

Pointer<LMesh> mesh = l_data_manager->getLMesh(finest_ln);
std::vector<LNode*> nodes = mesh->getLocalNodes();
nodes.insert(nodes.end(), mesh->getGhostNodes().begin(), mesh->
  getGhostNodes().end());

for (LNode* const node : nodes)
{
    IBSpringForceSpec* const spring_spec = node->getNodeDataItem<
  IBSpringForceSpec>();
    if (spring_spec == nullptr) continue;   // not the master of any
   spring

    // spring[0] is the stiffness and spring[1] is the rest length.
  The &
    // makes "spring" a reference to the numbers stored in the spec.
    for (std::vector<double>& spring : spring_spec->getParameters())
    {
        spring[1] += growth_rate * dt;
    }
}
```

"The resting lengths of a 2D fiber are increased at each time step. New files are `update_springs.C` and `update_springs.h`. The idea is similar to `update_target_point_positions`." (2015–2020; the course's `Example_2DSprings`.)

- the loop is the target-point loop with `IBSpringForceSpec` in place of the target spec;
- a point without a spec is not the *master* of any spring (the last point of the fibre); it is still the far end of one;
- `getParameters()` is a vector of vectors, one per spring the point masters; the inner loop changes every spring of the point. The course version changed only `[0]`, which is the same thing on a fibre where each point masters one spring, and wrong on a network;
- the reference `std::vector<double>& spring` is the whole point: next slide.

# THE BUG THAT WAS IN THE COURSE CODE FOR YEARS

```
// the 2015-2020 slides and the course's update_springs.C
double resting_length = spring_spec->getParameters()[0][1];
// "Get the resting lengths of the springs associated with that
  node."
resting_length += 0.01*dt;
// "increase the resting length of each spring with speed 0.01."
```

`double resting_length = ...` makes a copy of the number. Adding to the copy changes nothing in the spec, and `IBStandardForceGen` keeps reading the old rest length: the springs never grow, the run finishes, and nothing is reported. Every course copy of the example from 2013 to 2020 had this, including the slides; one 2014 edit (“Change 2-14”) fixed it in the code but kept the old lines as comments, so later copies reverted.

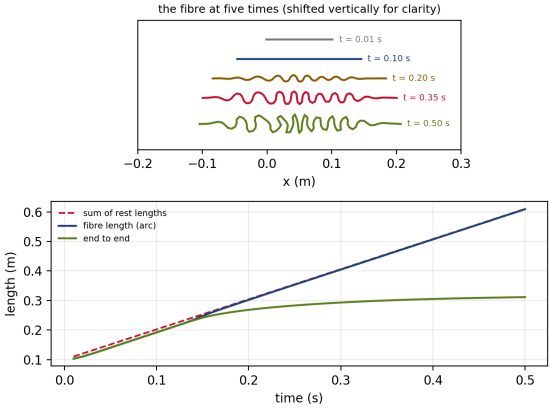
```
// the fix: change the stored number through a reference
std::vector<double>& params = spring_spec->getParameters()[0];
params[1] += 0.01*dt;
// or, for every spring of the point, the loop of the previous
  slide
```

How to tell, and how to avoid it:

- Tell: the fibre in the course’s run did not lengthen; the 2015–2020 “update_springs example” picture (a straight fibre with two symmetric eddies) is what a fibre that does not grow looks like. The series’ run (next slide) lengthens six-fold in 0.5 s.
- Avoid: whenever a getter returns a reference (slide 5), take it as a reference (`T&` or `auto&`); a plain `double` or `auto` copies. Check the header: the accessors come in `const` and non-`const` pairs for exactly this use.
- Verify: a short run and the fibre’s length against the sum of rest lengths (the figure of the next slide), or `pout` one rest length every few hundred steps.

The 2016 “stretching and shrinking plates” exercise (slide 27) had the same line, in one version written as `resting_length[0] += ...`, which does not compile with `getParameters()`.

# THE GROWING FIBRE: WHAT HAPPENS




A straight fibre of 103 points, springs only (no beams, no targets), $K = 2000$, in fluid of viscosity $100$. Every rest length grows at $0.01$ m/s, so the fibre's rest length grows at $1.02$ m/s (102 springs in series, chapter 8b's arithmetic).

- for the first $0.15$ s the fibre stays straight and lengthens nearly as fast as its rest lengths grow, pushing fluid out of the way; the springs are compressed by the drag (the arc length lags the rest-length sum by up to 7%);
- then it buckles: with no bending stiffness, a compressed fibre has no straight equilibrium, and from $0.15$ s on it folds into waves whose wavelength is set by the fluid's resistance, not by the structure;
- by $0.5$ s the rest lengths add up to $0.61$ m and the arc length is $0.609$ m (the springs are nearly relaxed), but the fibre spans only $0.31$ m end to end.

What to try (README): a slower `growth_rate` (less drag, straight for longer), a stiffer `kappa_spring` (smaller `DT`), or beams (chapter 10), which give the buckling a wavelength of their own.

IB/06 with IBAMR 0.19.0, shipped input (0.5 s); `tools/ch9a_figures.py`. The course version ran 2 s with `DT = 1e-6`; the series runs 0.5 s at `DT = 1e-4`.

# CHANGING OTHER THINGS IN TIME

The same loop, with other references:

```
// a spring's stiffness (parameter 0), e.g. activation that switches on at t_on
for (std::vector<double>& spring : spring_spec->getParameters())
    spring[0] = (current_time > t_on) ? K_active / ds : K_passive / ds;     // the FILE value, K/ds (chapter 6)

// a target point's stiffness or damping
double& kappa = force_spec->getStiffness();   kappa = ...;
double& eta   = force_spec->getDamping();     eta   = ...;

// release a tether: set its stiffness to zero (the point then moves freely with the fluid)
force_spec->getStiffness() = 0.0;
```

- IBFE/01, 05, 08 and 12 release their tethers at a given time in the IBFE way (the tether force law tests the time, chapter 15); for IB structures the equivalent is `getStiffness() = 0.0` in an update function.
- A rest length can be made to oscillate: `spring[1] += A * sin(2*PI*f*current_time) * dt` (the 2018 exercise of chapter 8b, with $A = 0.00049$ per spring for a plate that lengthens 50% in a second at 1 Hz).
- Changing a stiffness changes the stable time step (chapter 8a); changing a rest length does not, but a rest length that shrinks past zero turns the spring into a repulsive one with the wrong sign: keep it positive.
- Beams: `IBBeamForceSpec::getBendingRigidities()` and `getMeshDependentCurvatures()` are references too; chapter 10 changes the preferred curvature in time to make a fin flap.

# MULTIPLE BOUNDARIES: WHY

From the 2014–2020 lectures, word for word:

- “The advantage is that you can easily make them do different things.”
- “The disadvantage is that I don’t think you can connect springs between them.”
- “However, if you initialize boundaries in the same positions, they will stick together in those positions. This has the effect of gluing different boundaries or parts of boundaries together.”
- “You can make more than one boundary in IBAMR by creating additional `.spring`, `.vertex`, `.target` files. When you do this, you must add them to the input file.”

> Why the glue works: two points at the same place interpolate the *same* fluid velocity (chapter 5) and so move together for ever, exactly, without any force between them. It is a kinematic weld, not a joint: it transmits displacement, and the forces each structure applies are simply added onto the same fluid. It fails only if the two points are ever at different places, which the interpolation prevents (up to round-off, which IBAMR does not accumulate because both see the same velocity field).

Slide 28 says what 0.19.0 does and does not allow between structures. First, the mechanics.

# MULTIPLE BOUNDARIES: THE INPUT BLOCK AND THE GLOBAL NUMBERING

```
IBStandardInitializer {
   max_levels = MAX_LEVELS
   structure_names = "plate2d_left_512" , "
  plate2d_rght_512"
   plate2d_left_512 {
      level_number = MAX_LEVELS - 1         // the
   finest grid level
   }
   plate2d_rght_512 {
      level_number = MAX_LEVELS - 1
   }
}
```

```
// update_target_point_positions.cpp (IB/07): one range per
  structure
const std::pair<int, int> plate2d_left_lag_idxs = l_data_manager->
  getLagrangianStructureIndexRange(0, finest_ln);
const std::pair<int, int> plate2d_rght_lag_idxs = l_data_manager->
  getLagrangianStructureIndexRange(1, finest_ln);
```

- each structure has its own set of files (`plate2d_left_512.vertex`, `.spring`, `.beam`, `.target`, and the same for `rght`) and its own sub-block with a `level_number` (chapter 6);
- the order of `structure_names` is the order of the structures: structure 0, structure 1, . . . , and the order of the global point numbers. Point numbers start at 0 in each structure's files; IBAMR numbers all points together in that order, so the left plate's points are 0–102 and the right plate's are 103–205 (README).

The index ranges that tell them apart: next slide.

# MULTIPLE BOUNDARIES: THE INDEX RANGES

- `getLagrangianStructureIndexRange(k, ln)` returns structure $k$'s range of global indices on level `ln`: "`.first` is the first Lagrangian index for the left plate, `.second` is" one past "the last". (The 2020 slide said "the last"; the header says half-open, and the code's `< second` test agrees.)
- "You will need these to determine which boundary you are on": the test `first <= lag_idx && lag_idx < second` inside the loop routes each point to its own motion (slide 25).
- Names are also available: `getLagrangianStructureNames(ln)` lists them in order, and `getLagrangianStructureIndex(name, ln)` returns the number of a named structure, if you would rather not count on the order of `structure_names`.
- The ranges are per *level*: a structure on a coarser level (`level_number` less than the finest, chapter 6) is not in the finest level's mesh at all, and the update function must ask for that level's mesh and ranges instead. The examples put every structure on the finest level.
- In parallel every process knows every range (they are global numbers); the loop over local and ghost nodes then visits only the points it holds.

> The ranges are also how the `X.*` and `F.*` files of chapter 11 are read: the left plate's 103 points come first, then the right's, in global order.

# MOVING TWO PLATES APART (IB/07)

```
if (current_time <= time_to_acc)
{
    if (plate2d_left_lag_idxs.first <= lag_idx && lag_idx <
  plate2d_left_lag_idxs.second)
        X_target[0] -= current_time / time_to_acc * V * dt;
    if (plate2d_rght_lag_idxs.first <= lag_idx && lag_idx <
  plate2d_rght_lag_idxs.second)
        X_target[0] += current_time / time_to_acc * V * dt;
}
else
{
    if (plate2d_left_lag_idxs.first <= lag_idx && lag_idx <
  plate2d_left_lag_idxs.second)
        X_target[0] -= V * dt;
    if (plate2d_rght_lag_idxs.first <= lag_idx && lag_idx <
  plate2d_rght_lag_idxs.second)
        X_target[0] += V * dt;
}
```

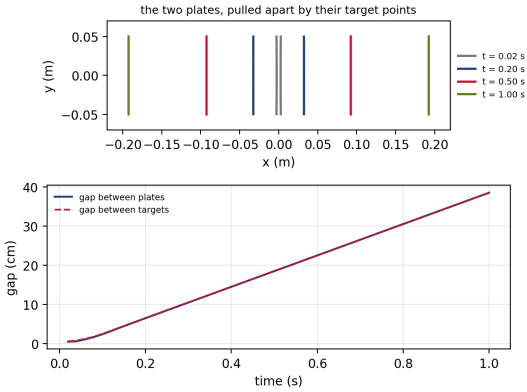
"This code will check to see if you are on the right plate or left plate, and then move the plate to the left or right, respectively." (2015–2020)

"This example considers two plates with bending and stretching stiffness that move apart as a result of moving target points" (`Example_2Dmoving_plates`, now IB/07).

- two vertical plates $0.1$ m long, $103$ points each, with springs ($K = 2000$), beams ($EI = 5 \times 10^{-3}$) and a target on every point ($\kappa = 2000/\Delta s$); they start $5$ mm apart, $2.6$ cells;
- the left plate's targets move left, the right plate's right, with the same ramp as the cylinders ($V = 0.2$ m/s after $0.1$ s);
- the course ran 2 s, by which time each plate had moved $0.39$ m and the two met again through the periodic boundary, $0.22$ m apart; the series runs 1 s.

Because they start only $2.6$ cells apart, "the two plates push and pull the same fluid" at first (README): the kernel's four-cell footprints overlap, and the flow between them is the sum of both. That is the glue of slide 22 at work, briefly, between structures that are *not* coincident.

# THE TWO PLATES: WHAT HAPPENS




Top: the plates at four times; bottom: the gap between them against the gap between their targets.

- the gap grows from $0.5$ cm to $38.5$ cm in 1 s, and the two curves are indistinguishable: the plates follow their targets to four digits (the targets' gap is $0.005 + 2 \times 0.19 = 0.385$ m at $t = 1$ s);
- the plates stay straight and vertical: every point is tethered, so the beams and springs have nothing to do. Tether only some points (the README's suggestion: write fewer lines into the `.target` files) and the plates bend as they are dragged, like the leaf of chapter 8a;
- the fluid between them is sucked in as they separate and pushed out ahead of them (the 2015–2020 picture, redrawn in chapter 26 from the `viz_IB2d` output).

`DT = 1e-4` runs; `5e-5` gives the same gap and forces to 4 digits at $t = 0.2$ s; `2e-4` stops after 7 steps with `Time step size change encountered` (README).

IB/07 with IBAMR 0.19.0, shipped input (1 s); `tools/ch9a_figures.py`. The course version ran 2 s with `DT = 1e-6`.

# STRETCHING AND SHRINKING PLATES (THE 2015–2016 EXERCISE)

"You can comment out where you move the plates in `update_target_point_positions`, and remove the comment from `update_springs`. The code in `update_springs` will make one plate stretch and the other shrink."

```
// the exercise's code, with the copy bug fixed (a reference, not
   a copy)
for (std::vector<double>& spring : spring_spec->getParameters())
{
    if (plate2d_left_lag_idxs.first <= lag_idx && lag_idx <
  plate2d_left_lag_idxs.second)
        spring[1] += 0.01 * dt;   // "you could make the left
  plate grow ..."
    if (plate2d_rght_lag_idxs.first <= lag_idx && lag_idx <
  plate2d_rght_lag_idxs.second)
        spring[1] -= 0.01 * dt;   // "... and the right shrink"
}
```

The 2016 slide "Left stretch, right shrink" showed the result: the left plate longer than the right, each with its pair of eddies. The figure is not in the GitHub copy of the lecture (the crosswalk notes it); chapter 26's VisIt recipe reproduces it from a run.

"Move and stretch? You do not want to move the boundaries with target points and stretch/shrink the plates at the same time. Why would this be problematic?" (the 2015–2020 question.)

The answer: every point of each plate is tethered to a target that moves as a rigid body. If the springs then try to lengthen the plate, each point is pulled one way by its spring and the other by its tether, and the tether wins ($\kappa = 2000/\Delta s$ against $K/\Delta s$ with $K = 2000$: the same number, so they fight to a standstill, with forces of either sign on neighbouring points). The plate neither moves cleanly nor stretches, the fluid gets a noisy force field, and the time step may fail. To do both, tether *one* point (or a few) and let the springs set the shape; or move the targets apart along the plate at the same rate as the rest lengths grow, which is the same thing said twice.

The shrinking plate has its own trap: a rest length that reaches zero. At $0.01$ m/s per spring and $\Delta s \approx 10^{-3}$ m that happens at $t = 0.1$ s; after that the springs push apart (slide 21). The course ran it for 2 s.

# CAN STRUCTURES BE CONNECTED? WHAT 0.19.0 ALLOWS

The 2014–2020 lectures said “I don’t think you can connect springs between them.” Checked against the initializer:

- A `.spring` (or `.beam`, `.xspring`) file names points by their index *within that structure’s* `.vertex` file; `IBStandardInitializer` adds the structure’s offset when it builds the global numbering. There is no syntax for “point 5 of the other structure”, so files cannot join two structures. The statement was right.
- Three ways around it:
  1. one structure, several parts: put both plates in one `.vertex` file and join them with springs in the one `.spring` file; use the index ranges you know (points 0–102 are the left plate) in the update function instead of `getLagrangianStructureIndexRange`;
  2. the glue: coincident points (slide 22), when the joint is a weld;
  3. code: `IBRedundantInitializer` (chapter 6) builds structures and their springs in `main.cpp` with explicit global indices, and a custom force (chapter 9b) can act between any two points.

# SEVERAL STRUCTURES OR ONE WITH PARTS?

| | several structures | one structure with parts |
|---|---|---|
| files | one set per structure, listed in `structure_names` | one set; the parts are index ranges you know from the generator |
| springs and beams between the parts | impossible through files | ordinary lines of the `.spring` and `.beam` files |
| per-structure settings (`level_number`, `uniform_*`, `posn_shift`, `length_scale_factor`; chapter 6) | each has its own | shared |
| telling the parts apart in code | `getLagrangianStructure IndexRange(k, ln)` | your own index arithmetic (and the generator's comments) |
| VisIt (chapter 26) | one Lagrangian mesh per structure in `lag_data.visit` | one mesh |
| forces (chapter 11) | `integral{F}` is the sum over all; per-structure sums need the ranges | the same |

Use several structures when they are separate bodies with separate motions (two plates, a row of cylinders and a wall, a heart and its valves); use one structure with parts when they are joined and the joint carries force (a fin on a body, a valve leaflet on a tube). Coincident points (slide 22) join two structures without force, which is enough for a hinge that only transmits position.

# PRESCRIBED PATHS FROM DATA: INTERPOLATING BETWEEN TWO STATES

The 2015 MATH 769 lectures added two of Nick Battista's examples: "Moving a cylinder in 2D" (a complete process: create the geometry files, create coordinate `.txt` files to be read in by `update_target_point_positions.C`, change the code accordingly, `make main2d`, run) and "Moving (interpolating) between two states". Their zip files lived on a UNC web page that no longer exists; the method survives.

```
// the pattern: targets move from state A to state B over a time
   T
// X_A, X_B: vectors of positions read once from two files (
   static, filled on the first call)
const double s = std::min(current_time / T, 1.0);          // or
   a smooth ramp, 0.5*(1 - cos(pi*s))
X_target[0] = (1 - s) * X_A[lag_idx][0] + s * X_B[lag_idx][0];
X_target[1] = (1 - s) * X_A[lag_idx][1] + s * X_B[lag_idx][1];
```

Here the target is set, not shifted, so the function must know the two states; read them on the first call (a `static` flag) from files written by the same generator that wrote the `.vertex` file, in the same point order. The 2015 "moving a cylinder" example did exactly this with coordinate `.txt` files.

The two 2015 examples are the simplest form of data-driven kinematics: the structure does not compute its motion, it is told it, frame by frame. Notes on using the pattern: next slide.

# INTERPOLATING BETWEEN STATES: NOTES ON THE PATTERN

- A smooth $s(t)$ (the cosine ramp) avoids the impulsive start; a periodic $s(t) = \frac{1}{2}(1 - \cos 2\pi f t)$ makes the structure go back and forth between the states at frequency $f$: a flapping wing from two drawn positions, the simplest kinematics. The frequency is the Strouhal number's $f$ (chapter 8b).
- More than two states: interpolate between consecutive frames of a digitized motion, $\mathbf{X}_{\text{target}} = (1 - s)\mathbf{X}_k + s\mathbf{X}_{k+1}$ with $k$ and $s$ from the time; chapter 1b's geometry pipeline ends with exactly such files, one per frame, in one point order.
- The states must have the *same number of points in the same order*; a generator that writes all frames from one parametrization guarantees it. A digitized outline with a varying number of points must be resampled first.
- Setting targets rather than shifting them is robust to restarts (the position depends on the time only) and to a changed `DT`; shifting accumulates round-off and depends on the step history, which is harmless for a ramp but not for a long periodic motion.
- IB2d does this with its `update_target_point_positions` script in MATLAB or Python (chapter 4c); the logic is identical and a good place to prototype.

This is how the lab's insect-wing and jellyfish kinematics were driven (Part VI); the valveless-pump set-up of chapter 36 uses the shift form with a travelling wave.

# PARAMETERS: OUT OF THE CODE AND INTO THE INPUT FILE

The examples keep `V` and `time_to_acc` as `static const` numbers in the update function, as the course did. A sweep then means editing and rebuilding for every value. The better way, used by the pump set-up of chapter 36 and recommended there: read them from `input2d`.

```
// main.cpp: read once, pass down
const double V = input_db->getDouble("V_PLATES");
const double t_acc = input_db->getDouble("T_ACC");
...
update_target_point_positions(patch_hierarchy, l_data_manager,
  loop_time, dt, V, t_acc);
```

```
// input2d
V_PLATES = 0.2      // speed of the plates after
  the ramp (m/s)
T_ACC    = 0.1      // ramp time (s)
```

- add the two arguments to the function and its header; `input_db` is the top-level database `main.cpp` already holds (chapter 3);
- or pass the whole database (`Pointer<Database> db`) and let the function read what it needs;
- then a job array over `V_PLATES` (chapter 8a) needs no rebuild, and the value is recorded in the input file of each run instead of in a binary.

Either way, keep the motion's *units* in the comment: the speed is in m/s because the box is in metres; a dimensionless input (chapter 8b) would say "in units of $U$".

The same holds for `growth_rate` in `update_springs.cpp`, and for the frequency of any oscillating motion.

# TIME STEP AND STIFFNESS WITH MOVING TARGETS

A target point is a stiff spring, and a moving one must be stiff enough to drag the structure through the fluid without lagging (slide 15). The stiffness sets the time step (chapters 5 and 8a):

| example | $\kappa$ (file) | `DT` | measured (READMEs) |
|---|---|---|---|
| IB/04 cylinders | $4.1 \times 10^6$ | $2 \times 10^{-5}$ | the largest step that runs; $5 \times 10^{-5}$ stops within a few steps |
| IB/07 plates | $2.04 \times 10^6$ | $10^{-4}$ | $5 \times 10^{-5}$ identical to 4 digits; $2 \times 10^{-4}$ stops after 7 steps |
| IB/02 channel walls | $2.05 \times 10^6$ | $10^{-4}$ | fixed targets; runs |
| IB/08 fibre (3D) | $3.2 \times 10^4$ | $10^{-5}$ | fixed targets; with `TARGET_VELOCITY = 0.05` the run is unstable, $0.01$ runs |

- The lag is $F/\kappa$ and the step is $\propto 1/\sqrt{\kappa}$: a stiffness ten times smaller gives ten times the lag and three times the step. For a rigid body on a path, a lag of a tenth of a cell is usually fine; the cylinders are a thousand times stiffer than that.
- `error_on_dt_change = TRUE` turns a step the integrator cannot take into the abort `Time step size change encountered` (chapter 5): the symptom of a target too stiff for `DT`, or a target moved too far in one step.
- Damping ($\eta$, chapter 8a's estimate) tames a target point that rings after a sudden change of its path; the examples use none.

# CHECKING THAT A PRESCRIBED MOTION IS BEING FOLLOWED

Four checks, in order of cost:

1. Print: a `pout` of one target position and the corresponding point every few hundred steps (rank 0 only, or the output interleaves); the difference is the lag.
2. The force line: `integral{F}` (chapter 11) is the sum of the tether forces, which is the force the fluid exerts on the structure. For a body on a path it should look like drag: a ramp-up, then a plateau (IB/04: $249.6$ N/m at $0.2$ s); a force that grows without bound or oscillates at the step frequency means the targets are fighting something (slide 27) or the step is too large.
3. The positions: the `X.*` files against the path, as in the figures of this chapter (`tools/ch9a_figures.py`; chapter 11 has the reader). This is the check that catches the copy bug: a fibre whose length does not change.
4. VisIt: the `lag_data.visit` mesh at several times, and the velocity field around it (chapter 26); a structure that moves while the fluid does not is a tether without a kernel (wrong `level_number`, chapter 6).

> Do the first two on a two-step run before anything longer: a prescribed motion that is wrong in the first step is wrong in all of them, and a run of twenty thousand steps tells you nothing a run of two did not.

# WHAT CAN GO WRONG

| The mistake | What happens |
|---|---|
| a copy instead of a reference (`double r = ...[k][1]; r += ...`) | nothing changes, no error, no warning; the springs never grow (slide 19). The same for `Point X = ...` instead of `Point& X` |
| ghost nodes left out of the loop | correct on one process; in parallel, points near patch boundaries keep old targets and the structure tears or lags there (slide 11) |
| velocity added instead of displacement (`X_target[1] += V`) | the target jumps $V$ per step, $10^4$ times too far; the structure explodes or the run aborts at once |
| the wrong structure index | the other structure moves; or none, if the index does not exist (the range is empty) |
| the file not in `add_executable` | `undefined reference to `update_target_point_positions'` at link time; the header included but the function never built |
| edited the function, forgot to rebuild | the old motion runs; check the binary's time stamp or add a `pout` line |
| targets moved through a wall or out of the box | the points follow, and leave the domain: an abort at the next regrid (chapter 7) |
| $\kappa$ raised, `DT` not lowered | `Time step size change encountered` in the first steps (slide 33) |
| moving targets and changing rest lengths of the same points | the tethers and springs fight; noisy forces, no clean motion (slide 27) |
| a rest length driven below zero | the spring becomes repulsive; keep rest lengths positive (slide 21) |
| `getParameters()[0]` only | fine on a chain; on a network a master of several springs changes only its first (slide 18) |

# HOMEWORK

From the 2015–2020 courses and the READMEs (Appendix C collects the full set):

1. IB/04: make the row move sideways instead of up, and change `V` or `time_to_acc`; predict the lag from $\kappa$ and `integral{F}` before you measure it.
2. IB/07: move only one plate; then tether only the middle point of each plate and watch them bend as they are dragged.
3. IB/07: the stretch-and-shrink exercise with the corrected code; stop before the right plate's rest length reaches zero, and explain why "move and stretch" at once is a bad idea.
4. IB/06: make the rest length oscillate, `A*sin(2*PI*f*t)*dt` per spring, with the 2018 numbers ($A = 0.00049$, $f = 1$ Hz); then halve `growth_rate` in the original and report when the fibre buckles.
5. Write a function that moves a structure between two drawn states (slide 30) and drive IB/03's leaf with it instead of the stream.

> What to hand in (the 2018–2020 pattern): the changed lines of the update function, the `CMakeLists.txt` line, one VisIt frame per case, and the check of slide 34 that shows the motion was followed.

Extension: move `V` and `time_to_acc` into the input file (slide 32) and run a job array over three speeds (chapter 8a).

# RULES OF THUMB

- Target points prescribe a *force toward* a path, not a velocity; the lag is $F/\kappa$ and the price of a small lag is a small `DT`.
- Shift the targets by (speed at `current_time`) $\times$ `dt`, before `advanceHierarchy`; ramp the speed from zero.
- Loop over local *and* ghost nodes; test the spec pointer for `nullptr`; take every accessor as a reference.
- A spring belongs to its lower-numbered point; `getParameters()[k][0]` is the file stiffness, `[k][1]` the rest length; change every `k`.
- Several structures: one set of files each, listed in `structure_names`; that order is the numbering; the index range of each structure (`getLagrangianStructureIndexRange`) tells them apart.
- Files cannot join structures; one structure with parts, coincident points, or code can.
- Do not move targets and change rest lengths of the same points at once; keep rest lengths positive.
- Both source files in `add_executable`; rebuild after every edit; parameters into the input file for sweeps.
- Two steps, print one target and its point, read `integral{F}`, then the long run.

# SUMMARY

1. At run time each target point and each spring is a small object attached to its master point, holding $\kappa$, $\eta$, $\mathbf{X}_{\text{target}}$ or the spring parameters, with accessors that return references. Prescribed motion in IB is changing those numbers before each step.
2. The update function receives the hierarchy, the `LDataManager`, the time and the step; it loops over local and ghost nodes of the finest level, asks each for its spec, and shifts the target (or the rest length) by a rate times `dt`. `main.cpp` calls it before `advanceHierarchy`, and `CMakeLists.txt` lists the file.
3. The cylinders follow their targets to half a micrometre at $\kappa = 4 \times 10^6$ and `DT = 2e-5`; the plates' gap tracks the targets' to four digits; the fibre lengthens six-fold and buckles into waves once drag compresses it, which it did not do in the course's run because the rest length was changed on a copy.
4. Several structures are several sets of files in `structure_names`, numbered in that order; the index ranges tell them apart; files cannot join them, but one structure with parts, coincident points or code can.
5. Check with a print, the force line, the position files and VisIt, on a short run first.

Next, chapter 9b: forces you write yourself, custom spring laws through `registerSpringForceFunction`, applied external forces, and active pumping.

# WHAT CHANGED SINCE THE OLDER COURSE NOTES

- The API is the 0.19.0 one throughout (`getNodeDataItem`, `IBTK::Point`, `getParameters`, `LMesh` nodes, `.cpp`, CMake), with the 2009–2020 spellings in a table so that old code can be read; the 2009 listing from the 2012 deck is kept for that purpose.
- The copy-versus-reference bug of the course's `update_springs` (2013–2020) is named, explained and fixed; the series' IB/06 grows and buckles, and the figure shows it.
- Ghost nodes, the half-open index range, the structure numbering (0–102, 103–205) and the parameters-in-the-input-file pattern are new; so are the three measured figures, the lag table and the time-step table.
- "I don't think you can connect springs between them" is answered from the initializer, with the three ways around it; the "glue" of coincident points is explained; the "Move and stretch?" question is answered.
- The 2015 Battista examples (moving a cylinder, interpolating between two states) are kept as the method; their UNC download links are dead.
- The course comments that called the cylinders and plates "the wing" and said the row moves "left or right" are corrected in the examples; sakai, nemo and Google Code links are gone; the 2013 slide's server password is not reproduced.

# RECORDINGS (LISTED, NOT INCORPORATED)

The front matter's video list (chapter 0.1) has these recordings of this material:

| Recording | Date | Course |
|---|---|---|
| `5-Video-SpringsTargetsPart1.mp4`, `Part2.mp4` | 2018-02-14 | MATH 769 (IBAMR Part 1: Getting Started) |
| `6-Video_IBAMRTutorial_MultipleBoundaries_DomainSize_InputFile.mp4`, `Part2.mp4` | 2018-03-19 | same (also covers chapter 7's domain sizes) |
| `Lecture 5 - Moving immersed boundaries.mp4` | 2020-05-19 | 2020 Zoom lectures |
| `IBAMR-MovingTarget-ChangeSprings.mp4` | 2024-07-28 | 2024 IBAMR videos |

The 2018 recordings follow the 2017 "5 – Changing Target Positions and Springs" and "Multiple Boundaries" decks (with the copy bug on screen); the 2020 and 2024 ones follow the 2020 Lecture 5. The IBFE tethered-plate video of 2024 belongs to Part IV.

# REFERENCES

- IBAMR 0.19.0: `ibtk/include/ibtk/LDataManager.h` (the documented example of moving target points and of reading positions), `LMesh.h`, `LNode.h`, `LNodeIndex.h`; `include/ibamr/IBTargetPointForceSpec.h`, `IBSpringForceSpec.h`, `IBBeamForceSpec.h`; `src/IB/IBStandardInitializer.cpp` (structure numbering), `IBStandardForceGen.cpp` (the forces); `ibtk/include/ibtk/HierarchyIntegrator.h` (the preprocess callback). https://github.com/IBAMR/IBAMR
- IBAMR FAQ: https://github.com/IBAMR/IBAMR/wiki/FAQ; class documentation: https://ibamr.github.io/docs.
- The examples of this series: IB/04 moving cylinders, IB/06 growing springs, IB/07 moving plates (each with `update_*.cpp`, `.h`, `main.cpp`, `CMakeLists.txt` and a README with the measured time steps and motions); `tools/ch9a_figures.py` for the figures.
- B. E. Griffith's `moving_plates` example (2009), as reproduced in the January 2012 tutorial; now `examples/IB/explicit` in the IBAMR tree holds the current moving-structure examples (chapter 15).
- Sources: *Immersed boundary tutorial continued* (January 2012); *Changing spring lengths, stiffnesses and target points* and *IBAMR with multiple boundaries* (MATH 769, February 2013); *MATH 769 Lecture 8* and *Lecture 12* (2015); *4 – Changing Target Positions and Springs* and *5 – Multiple Boundaries* (public folder, 2014–2016); *5 – IBAMR Tutorial: Changing Target Positions and Springs* (MBI, 2017); *Lecture 5 – IBAMR Tutorial: Changing Target Positions and Springs* (2020, edited 2024).